

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN



PROYECTO 4: Sistema de exploración y búsqueda en un entorno jerárquico

Proyecto de Unidad

Integrantes: Dennis Patricio Criollo Chushig
Cristopher Daniel Lanchimba Carvajal
Didier Robin Elbay Enriquez
Felipe Sebastián Moreno Loza

Asignatura: Modelos Discretos

Julio 2026

1. Introducción

Los sistemas de archivos y, en general, cualquier estructura de datos organizada de forma jerárquica, constituyen uno de los modelos más frecuentes e importantes en la informática moderna. Carpetas que contienen subcarpetas y archivos, menús multinivel de aplicaciones, organigramas empresariales, árboles de decisión en inteligencia artificial o la propia estructura de directorios de un sistema operativo son ejemplos cotidianos de entornos jerárquicos. En todos estos contextos, la capacidad de explorar el espacio de forma sistemática, eficiente y controlada, así como de localizar un elemento concreto, resulta fundamental tanto desde el punto de vista teórico como práctico.

El presente proyecto aborda el desarrollo de un sistema computacional que modela la exploración de un entorno jerárquico compuesto por múltiples niveles, subniveles y rutas posibles. Cada elemento del entorno puede conducir a distintas opciones: algunas permiten continuar la exploración (carpetas) y otras representan puntos finales o bloqueos (archivos). El sistema permite recorrer este entorno partiendo desde un punto inicial y analizar distintas estrategias de exploración según criterios definidos por el grupo de trabajo.

Para cumplir con este propósito se implementaron dos algoritmos clásicos de recorrido de grafos y árboles: la búsqueda en profundidad (*Depth-First Search*, DFS) y la búsqueda en anchura (*Breadth-First Search*, BFS). Ambos algoritmos permiten localizar un archivo objetivo dentro de una estructura de directorios simulada, registrar el camino recorrido paso a paso y comparar la eficiencia de cada estrategia en términos de número de nodos visitados y orden de exploración.

El resultado final es un sistema funcional escrito en el lenguaje C++ que carga la estructura jerárquica desde un archivo de texto externo, ejecuta ambas búsquedas de manera independiente, muestra los resultados por consola y genera un reporte detallado en formato texto. De esta manera se evidencia que los estudiantes comprenden la estructura interna del sistema, la lógica de decisión implementada y la forma en que las decisiones tomadas en un nivel superior condicionan las opciones disponibles en los niveles inferiores.

Este trabajo se enmarca dentro de la asignatura de Modelos Discretos y responde específicamente a los requerimientos del Proyecto 4 planteado por el Departamento de Ciencias de la Computación de la Universidad de las Fuerzas Armadas ESPE.

2. Objetivos

2.1. Objetivo general

Desarrollar un sistema computacional que modele la exploración de un entorno jerárquico y permita comparar estrategias de búsqueda (DFS y BFS) para localizar elementos específicos, registrando los caminos recorridos y justificando los resultados obtenidos en función de las decisiones tomadas durante el proceso.

2.2. Objetivos específicos

- Representar de forma clara y completa una estructura jerárquica mediante un árbol n-ario implementado con la técnica de punteros primer hijo – siguiente hermano.
- Cargar dinámicamente la estructura del entorno desde un archivo de texto externo, evitando la inclusión de datos fijos (hardcodeados) dentro del código fuente.
- Implementar el algoritmo de búsqueda en profundidad (DFS) de forma recursiva, registrando cada nodo visitado.
- Implementar el algoritmo de búsqueda en anchura (BFS) de forma iterativa utilizando colas, registrando igualmente el historial de exploración.
- Generar un reporte comparativo que muestre el número de nodos visitados y la secuencia exacta de caminos explorados por cada estrategia.
- Analizar y evidenciar cómo las decisiones tomadas en niveles superiores del árbol condicionan las opciones disponibles en los niveles inferiores.
- Demostrar la comprensión de la estructura interna del sistema y de la lógica de decisión implementada a través de un sistema funcional y documentado.

3. Marco teórico

3.1. Árboles como estructura de datos

Un árbol es un grafo conexo y acíclico. En su forma más general, conocida como árbol n-ario, cada nodo puede tener un número arbitrario de hijos. Esta estructura es especialmente adecuada para modelar jerarquías, ya que establece una relación clara de pertenencia entre un elemento padre y sus elementos hijos.

Existen varias formas de representar un árbol en memoria. Cuando el grado de cada nodo es variable (es decir, no se conoce de antemano cuántos hijos tendrá), la representación más eficiente y flexible es la técnica de “primer hijo – siguiente hermano” (*first-child next-sibling*). En esta representación cada nodo almacena únicamente dos punteros: uno que apunta a su primer hijo y otro que apunta a su hermano inmediato (el siguiente nodo que comparte el mismo padre). De esta forma, el árbol n-ario se convierte en una estructura binaria equivalente, facilitando tanto el recorrido como la gestión dinámica de memoria.

Esta técnica es ampliamente utilizada en la implementación de sistemas de archivos, compiladores (árboles de sintaxis abstracta) y en general en cualquier aplicación donde la cantidad de hijos por nodo no es fija.

3.2. Búsqueda en profundidad (DFS)

La búsqueda en profundidad (*Depth-First Search*) es un algoritmo que explora tan lejos como sea posible a lo largo de cada rama antes de retroceder (*backtracking*). Se implementa de forma natural mediante recursión (utilizando la pila de llamadas del sistema) o mediante una pila explícita.

En un árbol, DFS genera un recorrido que desciende completamente por una rama hasta llegar a una hoja o a un nodo sin hijos no visitados, y solo entonces regresa para explorar la siguiente rama disponible. Esta característica hace que DFS sea especialmente eficiente cuando el objetivo se encuentra en niveles profundos del árbol, ya que prioriza la profundidad sobre la amplitud.

Según el análisis realizado por Everitt y Hutter [1], cuando la probabilidad de encontrar el objetivo es mayor en niveles profundos, DFS tiende a localizarlo visitando menos nodos que BFS. Sin embargo, DFS no garantiza encontrar el camino más corto en número de aristas.

3.3. Búsqueda en anchura (BFS)

La búsqueda en anchura (*Breadth-First Search*) explora el árbol nivel por nivel. Utiliza una cola (estructura FIFO – First In, First Out) para garantizar que todos los nodos de un nivel se procesen completamente antes de pasar al siguiente nivel.

BFS garantiza encontrar el camino más corto en número de aristas cuando todas las aristas tienen el mismo peso. Esta propiedad lo convierte en el algoritmo preferido cuando se necesita la solución de menor profundidad. No obstante, su consumo de memoria puede crecer de forma significativa si el factor de ramificación del árbol es alto, ya que debe almacenar en la cola todos los nodos de un nivel antes de descender.

En el contexto específico de sistemas de archivos, Chen [3] señala que BFS puede resultar preferible en escenarios de red porque mantiene un número limitado de enumeraciones de directorios activas de forma simultánea, reduciendo el riesgo de que el servidor abandone conexiones antiguas por inactividad.

3.4. Comparación y aplicación a entornos jerárquicos

Diversos estudios han comparado DFS y BFS específicamente para la búsqueda de archivos en estructuras de directorios. Ginting y Sembiring [2] demuestran que ambos algoritmos reducen significativamente el tiempo de búsqueda respecto a una exploración manual, siendo la elección del algoritmo más adecuado dependiente de la profundidad esperada del archivo objetivo y del factor de ramificación de la estructura.

En términos de complejidad temporal, ambos algoritmos son $O(n)$ en el peor caso, donde n es el número de nodos del árbol, ya que en el peor escenario deben visitar todos los nodos. La diferencia radica en el orden de visita y en el consumo de memoria auxiliar: DFS utiliza $O(h)$

de memoria adicional (donde h es la altura del árbol), mientras que BFS puede requerir $O(w)$ (donde w es el ancho máximo del árbol).

4. Descripción del sistema

4.1. Modelo del entorno jerárquico

El entorno se modela como un árbol de directorios y archivos que simula una estructura típica de un sistema de archivos. La estructura de ejemplo utilizada en las pruebas es la siguiente:

C:

```
Universidad_ESPE
  Modelos_Discretos
    Automatas.pdf
  Proyectos_Personales
    Nomifi
      database.sqlite
  Juegos
    Clash_Royale
      estrategia.txt
  Windows
    System32
      cmd.exe
```

Esta estructura contiene 13 nodos en total (1 raíz, 4 carpetas de primer nivel, 4 subcarpetas y 4 archivos hoja). Cada línea del archivo de configuración `arbol.txt` tiene el formato:

```
Padre Hijo Tipo
```

donde `Tipo` es 0 para carpeta y 1 para archivo. El sistema lee este archivo línea por línea y construye dinámicamente el árbol en memoria, lo que permite modificar la estructura del entorno sin necesidad de recompilar el programa.

4.2. Estructura de datos

Se definió la siguiente estructura para representar cada nodo del árbol:

```
struct NodoArchivo {
    string nombre;
    bool esArchivo;
    NodoArchivo* primerHijo;
    NodoArchivo* siguienteHermano;
};
```

Las operaciones principales implementadas son:

- `crearNodo`: reserva memoria dinámica con `new`, asigna el nombre y el tipo del nodo, e inicializa ambos punteros a `NULL`.
- `agregarElemento`: inserta un nuevo nodo como último hijo de una carpeta existente. Si la carpeta no tiene hijos, el nuevo nodo se convierte en el `primerHijo`; en caso contrario, se recorre la lista de hermanos hasta llegar al final y se enlaza allí.
- `cargarArbolDesdeTXT`: abre el archivo de texto, lee cada tripleta (padre, hijo, tipo), busca o crea el nodo padre, crea el nodo hijo y los conecta mediante `agregarElemento`. El primer padre encontrado se establece como raíz del árbol.

Este diseño permite que el árbol crezca de forma completamente dinámica y que la misma lógica de construcción sirva para cualquier estructura jerárquica que se describa en el archivo de texto.

4.3. Algoritmos de búsqueda implementados

Búsqueda en profundidad (DFS) – versión recursiva:

El algoritmo DFS se implementó de forma recursiva. En cada llamada se realiza lo siguiente:

1. Se construye la ruta actual concatenando el nombre del nodo al camino recorrido hasta el momento.
2. Se registra la ruta completa en el vector de historial.
3. Se verifica si el nombre del nodo actual coincide con el objetivo buscado. En caso afirmativo se retorna `true`.
4. Se recorre de forma recursiva cada hijo del nodo actual (siguiendo el puntero `primerHijo` y luego `siguienteHermano`). Si alguna llamada recursiva retorna `true`, se propaga el éxito hacia arriba.
5. Si ningún hijo encuentra el objetivo, se retorna `false`.

Búsqueda en anchura (BFS) – versión iterativa con colas:

El algoritmo BFS se implementó de forma iterativa utilizando dos colas sincronizadas: una para los nodos y otra para las rutas de texto correspondientes.

1. Se encola la raíz junto con su nombre como ruta inicial.
2. Mientras la cola de nodos no esté vacía:
 - Se desencola el nodo actual y su ruta asociada.
 - Se registra la ruta en el historial.

- Si el nombre del nodo coincide con el objetivo, se termina la búsqueda.
- Se recorren todos los hijos del nodo actual y se encolan junto con la ruta actualizada (ruta del padre + “/” + nombre del hijo).

4.4. Registro y persistencia de resultados

Al finalizar ambas búsquedas, el sistema genera automáticamente el archivo `registro_busqueda.txt`. Este archivo contiene:

- Un encabezado identificativo del reporte.
- La sección de resultados de DFS, con el total de nodos visitados y la lista ordenada de cada paso del recorrido.
- La sección de resultados de BFS, con la misma información.

Este registro permite analizar de forma posterior y objetiva las diferencias entre ambas estrategias sin necesidad de volver a ejecutar el programa.

5. Implementación y resultados

El sistema se implementó íntegramente en C++ utilizando las bibliotecas estándar `iostream`, `string`, `vector`, `queue` y `fstream`. La compilación se realizó con el compilador g++ bajo un entorno Windows con MinGW. No se utilizaron librerías externas ni estructuras de datos de la STL más allá de las mencionadas, con el fin de evidenciar el control manual de la memoria y de los punteros.

5.1. Proceso de ejecución

Al iniciar el programa, este intenta cargar el árbol desde el archivo `arbol.txt`. Si el archivo no se encuentra, se muestra un mensaje de error y el programa termina. Una vez cargado el árbol, se solicita al usuario que escriba el nombre exacto del archivo que desea buscar. A continuación se ejecutan secuencialmente las dos búsquedas (primero DFS y luego BFS), mostrando por consola el resultado de cada una (si el archivo fue encontrado y cuántos nodos se visitaron). Finalmente se genera el archivo de registro.

5.2. Ejemplo de ejecución

Al buscar el archivo `cmd.exe` (ubicado en la última rama del árbol), ambos algoritmos visitan los 13 nodos existentes. El reporte generado muestra claramente la diferencia de orden de visita:

- **DFS:** desciende completamente por la rama `Universidad_ESPE` → `Modelos_Discretos` → `Automatas.pdf`, luego retrocede y explora `Proyectos_Personales`, después `Juegos` y finalmente `Windows` → `System32` → `cmd.exe`.

- **BFS:** visita primero todos los hijos directos de C : (Universidad_ESPE, Proyectos_Personales, Juegos, Windows), después todos los nietos (Modelos_Discretos, Nomifi, Clash_Royale, System32) y por último los archivos hoja.

5.3. Análisis comparativo de resultados

Cuando el archivo objetivo se encuentra en una rama temprana del árbol (por ejemplo `Automatas.pdf`), el comportamiento de ambos algoritmos cambia de forma notable:

- DFS localiza el archivo tras visitar únicamente 4 nodos, ya que desciende inmediatamente por la primera rama.
- BFS necesita recorrer primero todos los nodos del nivel 1 antes de poder descender al nivel 2 y encontrar el archivo, visitando por tanto un número mayor de nodos.

Este comportamiento confirma las predicciones teóricas de Everitt y Hutter [1]: la eficiencia relativa de DFS frente a BFS depende directamente de la profundidad esperada del objetivo. Si el objetivo tiende a estar profundo, DFS suele ser más eficiente en número de nodos visitados; si tiende a estar cerca de la raíz, BFS puede resultar más conveniente y además garantiza el camino más corto.

La visualización de los caminos registrados permite observar de forma clara cómo una decisión tomada en un nivel superior (elegir explorar una carpeta determinada) condiciona irrevocablemente el subárbol que será explorado a continuación. Esta característica cumple con el requisito explícito del proyecto de mostrar la influencia de las decisiones locales sobre el comportamiento global de la red o del entorno.

6. Discusión

El sistema desarrollado cumple de manera satisfactoria con todos los requisitos establecidos en el enunciado del Proyecto 4:

- Representa la estructura jerárquica completa de forma dinámica y configurable.
- Permite visualizar los recorridos realizados por cada estrategia de exploración.
- Muestra de forma explícita cómo las decisiones tomadas en niveles superiores condicionan las opciones disponibles en los niveles inferiores.
- Compara dos estrategias clásicas de exploración (DFS y BFS) bajo las mismas condiciones.
- Genera un registro justificable y persistente de los resultados obtenidos.

Una limitación actual del sistema es que el árbol de ejemplo utilizado es relativamente pequeño (13 nodos). En estructuras de mayor tamaño y mayor factor de ramificación, la diferencia de rendimiento y de consumo de memoria entre DFS y BFS se vuelve más significativa. En

particular, BFS puede requerir una cola cuyo tamaño crece de forma proporcional al ancho máximo del árbol, lo que en árboles muy anchos puede convertirse en un problema de memoria.

Otra posible mejora futura consiste en incorporar pesos a las aristas (por ejemplo, tamaño de archivo o tiempo de acceso) y utilizar algoritmos informados como A*, o bien permitir la búsqueda de múltiples objetivos en una sola ejecución. No obstante, el diseño actual, al separar claramente la estructura de datos de los algoritmos de búsqueda, facilita la incorporación de estas extensiones sin necesidad de reescribir el núcleo del sistema.

El uso de la representación primer-hijo/siguiente-hermano resulta especialmente adecuada para un lenguaje de bajo nivel como C++, ya que evita el overhead de contenedores dinámicos (como `vector` o `list`) asociados a cada nodo y otorga un control total sobre la gestión de memoria.

7. Conclusiones

Se ha implementado exitosamente un sistema de exploración y búsqueda en un entorno jerárquico que permite:

1. Cargar dinámicamente cualquier estructura de árbol desde un archivo de texto externo, sin necesidad de modificar ni recompilar el código fuente.
2. Ejecutar búsquedas mediante dos estrategias distintas (DFS recursivo y BFS iterativo) registrando el camino completo de exploración en cada caso.
3. Comparar ambas estrategias de forma objetiva y justificar los resultados obtenidos en función de las decisiones tomadas durante el proceso de exploración.

El proyecto demuestra la comprensión de conceptos fundamentales de la asignatura de Modelos Discretos: representación de árboles, recorridos sistemáticos, uso de recursión y de estructuras auxiliares (pilas implícitas y colas), así como la influencia de las decisiones locales sobre el estado global del sistema.

La separación clara entre la estructura de datos y los algoritmos de búsqueda facilita futuras extensiones del sistema, tales como la incorporación de pesos en las aristas, la implementación de heurísticas informadas o la simulación de múltiples usuarios explorando el entorno de forma concurrente. De esta forma, el trabajo no solo cumple con los requisitos planteados, sino que sienta las bases para desarrollos posteriores más complejos.

8. Referencias

- [1] Everitt, T., & Hutter, M. (2015). Analytical Results on the BFS vs. DFS Algorithm Selection Problem. Part I: Tree Search. En *AI 2015: Advances in Artificial Intelligence* (pp. 174–185). Springer. https://doi.org/10.1007/978-3-319-26350-2_14

- [2] Ginting, S. E., & Sembiring, A. S. (2019). Comparison of Breadth First Search (BFS) and Depth-First Search (DFS) Methods on File Search in Structure Directory Windows. *LOGIN*, 13(1). <https://doi.org/10.24224/login.v13i1.23>
- [3] Chen, R. (2005). Why is breadth-first searching better for file system tree walking? *The Old New Thing* (Microsoft Developer Blog). <https://devblogs.microsoft.com/oldnewthing/20050203-00/?p=36533>
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Capítulos sobre grafos y árboles).
- [5] Universidad de las Fuerzas Armadas ESPE. (2026). *P3_Exp1_Proyectos_202650*. Documento de especificación de proyectos de la asignatura Modelos Discretos.